LAB 5 / STATIC CODE ANALYSIS

Supplier Purchase Order Tracker

Sasyak Subudhi | PES1UG24AM256

27 October 2026

Verified results

Tool	Before	After
Pylint 4.1.3	2.41/10 | 22 messages	10.00/10 | 0 messages
Bandit 1.9.4	3 issues: high, medium, low	0 issues
Flake8 7.4.1	4 style findings	0 findings

Objective and method

Improve the assigned Python program using Pylint, Bandit and Flake8, fix at least four issues, eliminate Bandit and Flake8 findings, and achieve a Pylint score above 9.5.

The SRN matches row 19 of the assignment workbook. The original program and final program both produce the demo result "notifying warehouse about Acme" followed by "1 540". Analysis used Python 3.12.14 with the same tool versions before and after. No checks were suppressed.

Source and evidence

Assigned repository: https://github.com/SETAPESU26/L5_18_supplier_orders

Original revision: 9b72a43356daac383db067724cb5cdccd7dccd4b

Raw reports are in reports/before and reports/after. Each contains pylint_report.txt, bandit_report.txt and flake8_report.txt, plus stderr and exit-code metadata. The final Flake8 output is empty because it found no issues. All three final exit codes are zero.

Report scope: supplier_orders.py. Eight separate regression tests passed. See reports/tests.txt and reports/demo.txt for executed evidence.

Issues identified and fixed

Original source line numbers are shown below. High and medium Bandit risks are listed first. Other categories describe code quality or style.

Tool / code	Type	Lines	Problem and resolution
B605 / Bandit	High	21	Supplier input was concatenated into os.system(), allowing shell metacharacters to run commands. Replace the shell call with print(); supplier text is treated literally (final 35-37).
B608 / Bandit	Medium	11	String interpolation built an INSERT statement from supplier and item input. Quotes could break the query or alter its meaning. Bind supplier, item and quantity with SQLite ? placeholders (final 19-23).
B110 / Bandit; W0718 / Pylint	Low / quality	25-28	A broad except Exception followed by pass hid failed cleanup operations. Remove the catch so sqlite3 errors propagate; verify with a missing-table test (final 40-42).
W0603 / Pylint	Quality	6, 10, 12	The cumulative order quantity depended on mutable global state. Keep the total in main and explicitly pass the prior total to place_order (final 17-25, 56-59).
W0621 / Pylint	Quality	4, 9, 16, 24, 31	Function parameters shadowed the module-level connection. Create the connection inside a factory; pass a local connection explicitly and close it in main (final 7-14, 54-55).
W0613 / Pylint	Quality	16	remarks was accepted but never stored or used. Remove the unused parameter and update the demonstration call (final 28, 60).
C0103 / Pylint	Convention	9, 16, 20, 31	Mixed-case function names violated Python naming conventions. Rename functions and call sites to snake_case.
C0114, C0116 / Pylint	Convention	1, 9, 16, 20, 24, 31, 35	Module and public functions lacked docstrings. Add module and function docstrings describing purpose and return values.
C0301 / Pylint; E501 / Flake8	Style	5, 11, 32	SQL statements and query expressions exceeded line-length limits. Use parenthesized multiline expressions and adjacent SQL string literals, keeping lines within 79 characters.
E225 / Flake8	Style	12	order_total+=quantity lacked operator spacing. Use order_total += quantity (final 24).
C0209 / Pylint	Convention	11	The original string-formatting expression also triggered a formatting suggestion. Parameterized SQL removes the formatting expression; an f-string would retain the injection risk.

Verification and reflection

Behavior checks and implementation scope

Eight tests passed: original demo output; SQL-like supplier input stored literally; notification without shell execution; cleanup preserving pending orders; visible database errors; cumulative totals after cleanup; independent databases; and import without demo side effects.

Functions now use snake_case; the unused remarks argument was removed. main() passes cumulative quantity explicitly, manages a transaction and closes the connection. A main guard prevents work during import. Existing call sites and tests use the revised API. The database remains in memory.

Three-commit workflow

1. Bandit: parameterized SQL, shell removal and visible errors. 2. Pylint: docstrings, naming, local connection and explicit total. 3. Flake8: line wrapping, spacing and final submission evidence. Git history is preserved in the ZIP; inspect it with git log -3 --oneline.

The Git author is Sasyak Subudhi with an SRN-based local placeholder email, PES1UG24AM256@localhost. No remote push or LMS submission was performed.

Reflection prompts

The supplied three-page handout does not list the four questions referenced by the README. These prompts cover its learning outcomes; they are not quoted official questions.

1. How do the three tools complement one another?

Pylint identifies maintainability and likely programming problems, such as global state, shadowed names, unused arguments and missing documentation. Bandit focuses on security patterns, including SQL construction and shell execution. Flake8 checks Python style and related coding mistakes. Their findings overlap, but none covers the entire program.

2. Which issues should be fixed first, and why?

The high-severity shell-injection finding B605 takes priority, followed by the medium-severity SQL-injection finding B608. Both involve interpreting supplier-controlled text as executable syntax. Printing notifications and binding SQL parameters remove those interpretation steps. Silent exception handling should then be removed so failures remain visible.

3. How were the changes verified?

All three tools were run against the original file and the final file with the same versions. Pylint improved from 2.41/10 to 10.00/10; Bandit fell from three findings to zero; Flake8 fell from four findings to zero. Eight regression tests check demonstration output, SQL payload handling, notification behavior, cleanup, visible errors, cumulative totals, independent databases and safe imports.

4. What are the limitations and lessons of static analysis?

A clean tool report does not prove correctness or security. Static tools did not directly flag the unguarded main() call or the missing explicit connection close; those were found by inspection. Runtime tests were needed to verify behavior. The practical lesson is to combine static analysis, source review and targeted tests, and to fix the underlying problem instead of suppressing warnings.